

Name: Abhishek Chaurasiya

Class: D15B

Roll No: 09

Case Study

Static Analysis Integration with Terraform

- Concepts Used: Jenkins, SonarQube, Terraform.
- Problem Statement: "Use Terraform to set up the infrastructure for a Jenkins server and a SonarQube instance. Then, configure a Jenkins pipeline to perform a static analysis of a Python application using SonarQube."
- Tasks:
 - Write a Terraform script to deploy Jenkins and SonarQube on AWS.
 - Set up a Jenkins pipeline to analyze a Python codebase using SonarQube.
 - Trigger the pipeline and review the analysis results in SonarQube.

Step-by-Step Explanation & Performance

Step 1

Setting Up the EC2 Instances with Terraform

To set up the infrastructure, Terraform scripts were written to create two EC2 instances one for Jenkins and one for SonarQube. The following steps outline the Terraform process:

Install Terraform:

Ensure that Terraform is installed on your local machine.

Create Terraform Configuration:

Define the resources in the main.tf file. The configuration specifies instance types, AMIs, security groups, and other necessary parameters.

Terraform Script for Jenkins Instance

```
# Jenkins EC2 Instance
resource "aws_instance" "jenkins" {
  ami           = "ami-0c7217cddde317cfec" # Amazon Linux 2 AMI
  instance_type = var.instance_type
  subnet_id     = aws_subnet.public.id

  vpc_security_group_ids = [aws_security_group.jenkins_sg.id]

  key_name = "adv" # Specify your existing RSA key pair name

  user_data = <<-EOF
  #!/bin/bash
  sudo yum update -y
  sudo wget -O /etc/yum.repos.d/jenkins.repo https://pkg.jenkins.io/redhat-stable/jenkins.repo
  sudo rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io-2023.key
  sudo yum upgrade
  sudo amazon-linux-extras install java-openjdk11 -y
  sudo yum install jenkins -y
  sudo systemctl enable jenkins
  sudo systemctl start jenkins
  EOF

  tags = {
    Name = "jenkins-server"
  }
}
```

Terraform Script for SonarQube Instance

```
# SonarQube EC2 Instance
resource "aws_instance" "sonarqube" {
  ami           = "ami-0c7217cdde317cfec" # Amazon Linux 2 AMI
  instance_type = var.instance_type
  subnet_id     = aws_subnet.public.id

  vpc_security_group_ids = [aws_security_group.sonarqube_sg.id]

  key_name = "adv" # Specify your existing RSA key pair name

  user_data = <<-EOF
  #!/bin/bash
  sudo yum update -y
  sudo yum install java-11-openjdk-devel -y
  sudo yum install wget unzip -y
  sudo wget https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-9.9.0.65466.zip
  sudo unzip sonarqube-9.9.0.65466.zip
  sudo mv sonarqube-9.9.0.65466 /opt/sonarqube
  sudo useradd sonar
  sudo chown -R sonar:sonar /opt/sonarqube
  sudo echo "sonar - nofile 65536" >> /etc/security/limits.conf
  sudo echo "sonar - nproc 4096" >> /etc/security/limits.conf
  sudo echo "/opt/sonarqube/bin/linux-x86-64/sonar.sh start" >> /etc/init.d/sonar
  sudo chmod 755 /etc/init.d/sonar
  sudo chkconfig --add sonar
  sudo service sonar start
  EOF

  tags = {
    Name = "sonarqube-server"
  }
}
```

Apply Terraform Configuration: Run the command terraform init followed by terraform apply to provision the instances.

```
C:\ABHI\CODE\Abhi\AdvDevops\terraform
λ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v5.73.0...
- Installed hashicorp/aws v5.73.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

After the above steps ,Go to AWS Instances and we can see our instances.

Instances (3) Info

Last updated 24 minutes ago

Find Instance by attribute or tag (case-sensitive)

All states ▾

☐	Name	Instance ID	Instance state ▾	Instance type ▾	Status check
☐	Jenkins	i-06484bace0d48b342	Stopped	t3.medium	–
☐	SonarQube	i-0ab5e94d5fa976f6b	Stopped	t3.medium	–
☐	sonarqube	i-0f468e1a794cf4d8d	Stopped	t2.micro	–

Step 2

Installing Jenkins and SonarQube

After provisioning the instances, Jenkins and SonarQube were installed as follows:

For Jenkins Installation:

SSH into the Jenkins EC2 instance.

```

λ ssh -i "adv.pem" bitnami@ec2-3-227-251-164.compute-1.amazonaws.com
Linux ip-172-31-76-173 6.1.0-26-cloud-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.112-1 (2024-09-30) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.

  _ _ _ _ _
 | _ | _ | _ | _ | _ | _ | _ | _ |
 | _ | _ | _ | _ | _ | _ | _ | _ |
 | _ | _ | _ | _ | _ | _ | _ | _ |

*** Welcome to the Bitnami package for Jenkins 2.462.3-2 ***
*** Documentation: https://docs.bitnami.com/aws/apps/jenkins/ ***
*** https://docs.bitnami.com/aws/ ***
*** Bitnami Forums: https://github.com/bitnami/vms/ ***
bitnami@ip-172-31-76-173:~$ curl localhost:8080
<html><head><meta http-equiv='refresh' content='1;url=/login?from=%2F'><script id='redirect' data-redirect-url='/login?
from=%2F' src='/static/637593c4/scripts/redirect.js'></script></head><body style='background-color:white; color:white;'>
Authentication required
<!--
-->

</body></html>

bitnami_credentials  htdocs  stack
bitnami@ip-172-31-76-173:~$ ls

```

Install Java and Jenkins using package managers. Follow these commands and run this in your SSH in Jenkins Instance.

```

sudo yum install -y java-1.8.0-openjdk
sudo wget -O /etc/yum.repos.d/jenkins.repo https://pkg.jenkins.io/redhat/jenkins.repo
sudo rpm --import https://pkg.jenkins.io/redhat/jenkins.io.key
sudo yum install -y jenkins
sudo systemctl start jenkins

```

For SonarQube Installation:

SSH into the SonarQube EC2 instance.

```
λ ssh -i "adv.pem" ec2-user@ec2-3-80-75-38.compute-1.amazonaws.com
Last login: Tue Aug 29 07:30:46 2023 from 44.226.56.253
```

```
  __|  __|_ )
 _| (  /   Amazon Linux 2 AMI
---|\____|
```

```
https://aws.amazon.com/amazon-linux-2/
40 package(s) needed for security, out of 69 available
Run "sudo yum update" to apply all updates.
[ec2-user@ip-172-31-20-59 ~]$ curl
curl: try 'curl --help' or 'curl --manual' for more information
[ec2-user@ip-172-31-20-59 ~]$ curl http://localhost:9000

<!DOCTYPE html>
<html lang="en">

<head>
  <meta http-equiv="content-type" content="text/html; charset=UTF-8" charset="UTF-8" />
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <link rel="apple-touch-icon" href="/apple-touch-icon.png">
  <link rel="apple-touch-icon" sizes="57x57" href="/apple-touch-icon-57x57.png">
  <link rel="apple-touch-icon" sizes="60x60" href="/apple-touch-icon-60x60.png">
  <link rel="apple-touch-icon" sizes="72x72" href="/apple-touch-icon-72x72.png">
  <link rel="apple-touch-icon" sizes="76x76" href="/apple-touch-icon-76x76.png">
  <link rel="apple-touch-icon" sizes="114x114" href="/apple-touch-icon-114x114.png">
  <link rel="apple-touch-icon" sizes="120x120" href="/apple-touch-icon-120x120.png">
  <link rel="apple-touch-icon" sizes="144x144" href="/apple-touch-icon-144x144.png">
  <link rel="apple-touch-icon" sizes="152x152" href="/apple-touch-icon-152x152.png">
  <link rel="apple-touch-icon" sizes="180x180" href="/apple-touch-icon-180x180.png">
```

Install Java and download SonarQube.

Follow these commands and run this in your SSH on SonarQube Instance.

```
sudo yum install -y java-1.8.0-openjdk
```

```
wget
```

```
https://binaries.sonarsource.com/Distribution/sonarqube/sonarqube-10.2.0.77676.zip
```

```
sudo unzip sonarqube-10.2.0.77676.zip -d /opt/
```

```
cd /opt/sonarqube-10.2.0.77676/bin/linux-x86-64/
```

```
sudo ./sonar.sh start
```

Step 3

Deploying the Python Project

With Jenkins and SonarQube running, the next step was to deploy a Python project:

Configure Jenkins Job:

New Item

Enter an item name

Select an item type



Freestyle project

Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.



Pipeline

Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.



Multi-configuration project

Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.



Folder

Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.



Multibranch Pipeline

Creates a set of Pipeline projects according to detected branches in one SCM repository.



Organization Folder

Creates a set of multibranch project subfolders by scanning for repositories.

OK

Create a new pipeline job in Jenkins

Use the following Jenkinsfile to clone the GitHub repository and trigger SonarQube analysis.

Pipeline

Definition

Pipeline script

Script ?

```
1 pipeline {
2   agent any
3   stages {
4     stage('Clone Repository') {
5       steps {
6         git branch: 'main', url: 'https://github.com/QuantumByteStudios/AdvDevOps'
7       }
8     }
9     stage('SonarQube Analysis') {
10      steps {
11        withSonarQubeEnv('sonarqube') {
12          sh '''
13            /opt/sonar-scanner/bin/sonar-scanner \
14            -D sonar.projectKey=YourProjectKey \
15            -D sonar.login=admin \
16            -D sonar.password=YourPassword \
17            -D sonar.host.url=http://<SonarQube-Instance-IP>:9000
18          '''
19        }
20      }
21    }
22  }
23 }
```

try sample Pipeline...

☒ Use Groovy Sandbox ?

[Pipeline Syntax](#)

```

pipeline {
  agent any
  stages {
    stage('Clone Repository') {
      steps {
        git branch: 'main', url: 'https://github.com/shazforiot/MSBuild_firstproject/'
      }
    }
    stage('SonarQube Analysis') {
      steps {
        withSonarQubeEnv('sonarqube') {
          sh '''
            /opt/sonar-scanner/bin/sonar-scanner \
            -D sonar.projectKey=YourProjectKey \
            -D sonar.login=admin \
            -D sonar.password=YourPassword \
            -D sonar.host.url=http://<SonarQube-Instance-IP>:9000
          '''
        }
      }
    }
  }
}

```

Dashboard > sonarqube >

Status

</> Changes
Workspace
Build Now
Configure
Delete Project
GitHub
SonarQube
Rename



sonarqube
Add description

Permalinks

- Last build (#3), 27 min ago
- Last stable build (#3), 27 min ago
- Last successful build (#3), 27 min ago
- Last failed build (#1), 9 days 12 hr ago
- Last unsuccessful build (#1), 9 days 12 hr ago
- Last completed build (#3), 27 min ago

Build History
trend

Filter...

#3
Oct 25, 2024, 9:00 AM

#2
Oct 15, 2024, 8:41 PM

#1
Oct 15, 2024, 8:39 PM

Dashboard > sonarqube > #3 > Console Output

Status

Changes

Console Output

Edit Build Information

Timings

Git Build Data

Previous Build

Console Output

Download

Copy

View as plain text

Started by user Harsh Jadhav

Running as SYSTEM

Building on the built-in node in workspace C:\ProgramData\Jenkins\jenkins\workspace\sonarqube

The recommended git tool is: NONE

No credentials specified

Cloning the remote Git repository

Cloning repository https://github.com/shazforiot/MSBuild_firstproject.git

> git.exe init C:\ProgramData\Jenkins\jenkins\workspace\sonarqube # timeout=10

Fetching upstream changes from https://github.com/shazforiot/MSBuild_firstproject.git

> git.exe --version # timeout=10

> git --version # 'git version 2.39.1.windows.1'

> git.exe fetch --tags --force --progress -- https://github.com/shazforiot/MSBuild_firstproject.git +refs/heads/*:refs/remotes/origin/* # timeout=10

> git.exe config remote.origin.url https://github.com/shazforiot/MSBuild_firstproject.git # timeout=10

> git.exe config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10

Avoid second fetch

> git.exe rev-parse "refs/remotes/origin/master^{commit}" # timeout=10

Checking out Revision f2bc042c04c6e72427c380bcae6d6fee7b49adf (refs/remotes/origin/master)

> git.exe config core.sparsecheckout # timeout=10

> git.exe checkout -f f2bc042c04c6e72427c380bcae6d6fee7b49adf # timeout=10

Commit message: "updated"

> git.exe rev-list --no-walk f2bc042c04c6e72427c380bcae6d6fee7b49adf # timeout=10

[sonarqube] \$ C:\ProgramData\Jenkins\tools\udson.plugins.sonar.SonarRunnerInstallation\sonarqube\bin\sonar-scanner.bat -Dsonar.host.url=http://localhost:9000 -Dsonar.projectKey=sonarqube-test -Dsonar.login=admin -Dsonar.host.url=http://3.80.75.38:9000 -Dsonar.password=password -Dsonar.projectBaseDir=C:\ProgramData\Jenkins\jenkins\workspace\sonarqube

14:30:42.059 WARN Property 'sonar.host.url' with value 'http://localhost:9000' is overridden with value 'http://3.80.75.38:9000'

14:30:42.073 INFO Scanner configuration file: C:\ProgramData\Jenkins\tools\udson.plugins.sonar.SonarRunnerInstallation\sonarqube\bin\..\conf\sonar-scanner.properties

Dashboard > sonarqube > #3 > Console Output

14:45:43.267 INFO o/v source files have been analyzed

14:45:43.267 INFO Sensor IaC Docker Sensor [iac] (done) | time=120ms

14:45:43.275 INFO ----- Run sensors on project

14:45:43.374 INFO Sensor C# [csharp]

14:45:43.375 WARN Your project contains C# files which cannot be analyzed with the scanner you are using. To analyze C# or VB.NET, you must use the SonarScanner for .NET 5.x or higher, see <https://redirect.sonarsource.com/doc/install-configure-scanner-msbuild.html>

14:45:43.375 INFO Sensor C# [csharp] (done) | time=1ms

14:45:43.375 INFO Sensor Analysis Warnings import [csharp]

14:45:43.377 INFO Sensor Analysis Warnings import [csharp] (done) | time=1ms

14:45:43.377 INFO Sensor C# File Caching Sensor [csharp]

14:45:43.377 WARN Incremental PR analysis: Could not determine common base path, cache will not be computed. Consider setting 'sonar.projectBaseDir' property.

14:45:43.377 INFO Sensor C# File Caching Sensor [csharp] (done) | time=0ms

14:45:43.377 INFO Sensor Zero Coverage Sensor

14:45:43.389 INFO Sensor Zero Coverage Sensor (done) | time=12ms

14:45:43.395 INFO SCM Publisher SCM provider for this project is: git

14:45:43.396 INFO SCM Publisher 4 source files to be analyzed

14:45:43.848 INFO SCM Publisher 4/4 source files have been analyzed (done) | time=450ms

14:45:43.853 INFO CPD Executor Calculating CPD for 0 files

14:45:43.854 INFO CPD Executor CPD calculation finished (done) | time=0ms

14:45:43.974 INFO Analysis report generated in 113ms, dir size=133.3 kB

14:45:44.033 INFO Analysis report compressed in 57ms, zip size=18.8 kB

14:45:45.616 INFO Analysis report uploaded in 1583ms

14:45:45.618 INFO ANALYSIS SUCCESSFUL, you can find the results at: <http://3.80.75.38:9000/dashboard?id=sonarqube-test>

14:45:45.618 INFO Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report

14:45:45.618 INFO More about the report processing at <http://3.80.75.38:9000/api/ce/task?id=42L8x6vI9jg6z43WJL6r>

14:45:45.632 INFO Analysis total time: 23.562 s

14:45:45.636 INFO EXECUTION SUCCESS

14:45:45.637 INFO Total time: 15:03.569s

Finished: SUCCESS

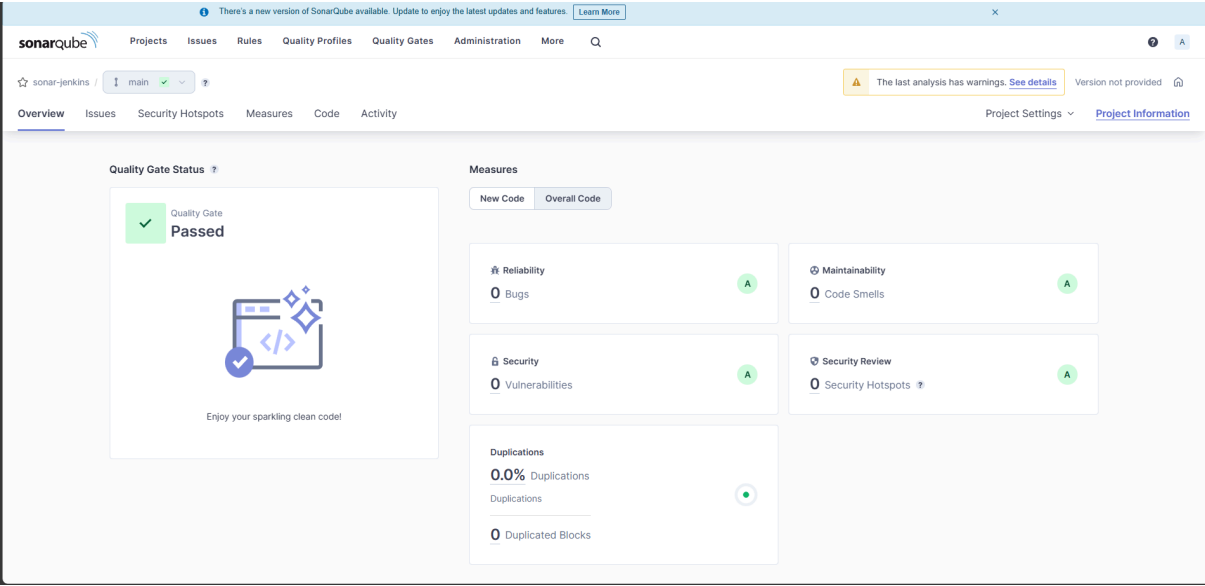
REST API Jenkins 2.462.2

Step 4

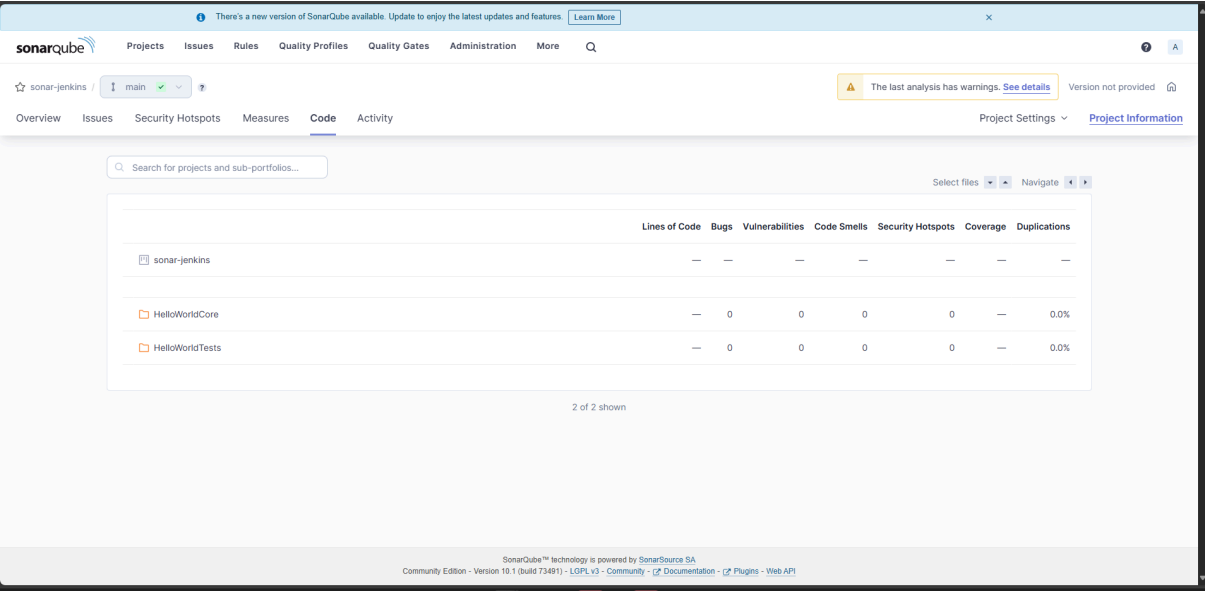
Analyzing Code Quality in SonarQube

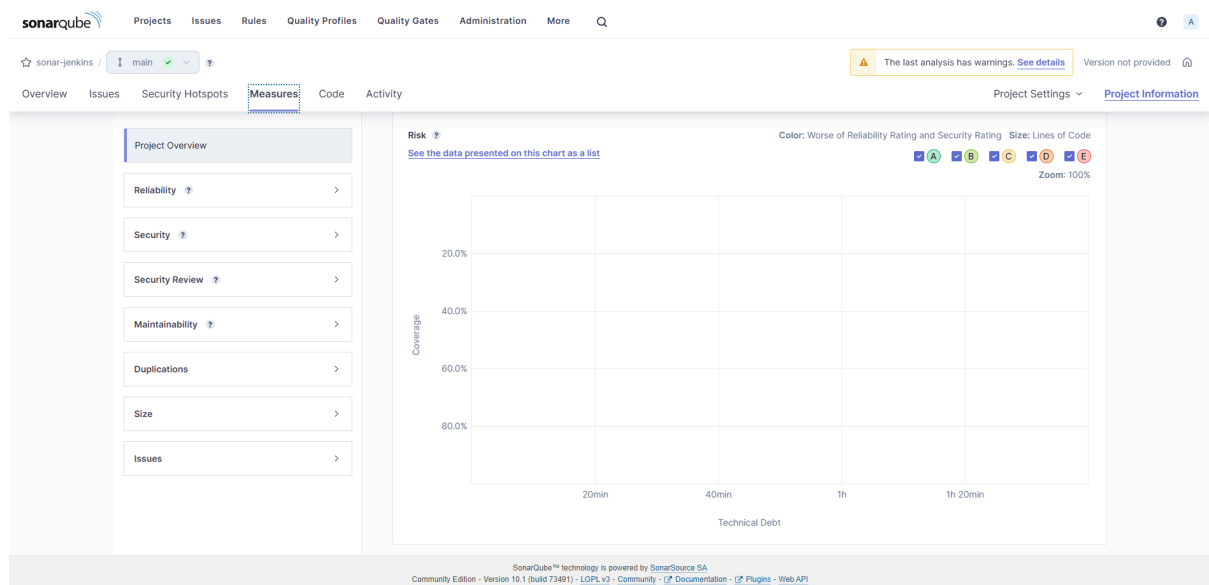
Once the project is deployed, SonarQube provides insights into the code quality

Create a Manual Project: In the SonarQube dashboard, create a new project and link it to the analysis performed by Jenkins.



Once the build is successfully completed, SonarQube automatically loads the project and displays the results, indicating a "Passed" status if no critical issues are found





Identify Code Smells: SonarQube identifies potential issues in the code, allowing developers to rectify them and maintain high coding standards.

Guidelines and Best Practices

1. Version Control Infrastructure

- Store Terraform scripts in a version control system like GitHub, GitLab, or Bitbucket to maintain a history of changes and ensure smooth collaboration among team members.

2. Secure Instances

- Use key pairs for SSH login instead of passwords to enhance security. Restrict access by only allowing trusted IP addresses to connect via SSH. Security Groups Configuration like
- Open port 8080 (TCP) for Jenkins to enable access to the web interface.
- Open port 9000 (TCP) for SonarQube to allow web-based project analysis and management.

3. Instance Type Recommendation

- Use t3.small or t3.medium instance types for Jenkins and SonarQube to ensure adequate CPU and memory, as both tools are resource-intensive. This will improve performance during large builds or code analysis operations.

4. Plugin Management

- Jenkins: Install essential plugins like:
 - Git Plugin for seamless integration with version control repositories.
 - Pipeline Plugin to manage CI/CD workflows effectively.
 - SonarQube Scanner Plugin for connecting Jenkins jobs with SonarQube for automated code analysis.
- SonarQube: Ensure the installation of relevant language analyzers or custom rules plugins to support the technologies used in your project. keep plugins updated to avoid compatibility issues

Demonstration Preparation

1. Key Points

- Overview of Deployment Process:
 - Begin with a concise overview of the entire deployment process. Discuss the Terraform scripts used to create the EC2 instances for Jenkins and SonarQube, emphasizing how Infrastructure as Code (IaC) simplifies and automates the deployment process.
 - Highlight the specific configuration details, such as selecting appropriate instance types (e.g., t3.small or t3.medium) based on the resource requirements of Jenkins and SonarQube. Discuss the significance of setting up security groups to allow necessary traffic (e.g., TCP on ports 8080 for Jenkins and 9000 for SonarQube).
- Integration of CI/CD and Code Quality Analysis:
 - Elaborate on how Jenkins facilitates Continuous Integration and Continuous Deployment (CI/CD) practices by automating the build, test, and deployment processes for software projects.
 - Explain the role of SonarQube in this workflow, highlighting its ability to perform static code analysis, detect vulnerabilities, and identify code

smells. Discuss how integrating these tools improves software quality and reduces time to market.

- Demonstration of Functionality:
 - Prepare to showcase a live demonstration of the end-to-end workflow, including deploying a sample Python project from a GitHub repository via Jenkins.

After the deployment, navigate to SonarQube to demonstrate how the project is analyzed, presenting the resulting metrics and any issues identified. Emphasize the insights provided by SonarQube, such as code coverage, technical debt, and maintainability ratings.

2. Practice

Rehearse the Demonstration:

- Conduct multiple practice runs of the demonstration to ensure fluency and confidence. Familiarize yourself with all commands, configurations, and expected outcomes to address potential technical issues effectively
- Focus on explaining each step clearly, so your audience understands the significance of the actions being taken.

Technical Setup Check:

- Prior to the demonstration, perform a thorough check of the technical setup. Ensure that the AWS EC2 instances for Jenkins and SonarQube are operational and that all necessary services are running.
- Verify that you can access the Jenkins UI and SonarQube dashboard seamlessly. Test the connectivity to your GitHub repository to ensure a smooth deployment process during the demonstration.

Questions:

- Clarify Technical Aspects:
 - The specific configurations made in the Terraform scripts, including instance types, security groups, and IAM roles.
 - Ans: In the Terraform scripts, specific configurations were made to ensure the proper setup of the infrastructure. Appropriate EC2 instance types, such as t2.micro, were selected for both Jenkins and

SonarQube based on resource requirements and cost considerations. Security groups were configured to allow inbound traffic, including port 8080 for Jenkins, port 9090 for SonarQube, and port 22 for SSH access, ensuring controlled connectivity. Additionally, IAM roles were assigned to the instances with the necessary permissions, enabling access to AWS services such as S3 and CloudWatch for logging or other operational needs. These configurations ensured a secure and functional deployment environment.

- The reasoning behind the choice of plugins installed in Jenkins (e.g., Git server, Pipeline, SonarQube Scanner) and how they enhance the CI/CD process.
 - Ans: The choice of plugins installed in Jenkins was driven by the need to streamline and enhance the CI/CD process. The Git plugin was installed to enable seamless integration with Git repositories, allowing Jenkins to pull code automatically whenever changes are committed. This ensures version control and facilitates automated builds. The Pipeline plugin was chosen to define and orchestrate CI/CD workflows as code, providing flexibility to create complex, multi-step pipelines for building, testing, and deploying applications. Additionally, the SonarQube Scanner plugin was installed to integrate Jenkins with SonarQube, automating the process of code quality analysis. This plugin ensures that code is continuously analyzed for bugs, vulnerabilities, and code smells during the build process, improving software quality.
- The steps taken to analyze the Python project in SonarQube, including any configurations made to the SonarQube project settings.
 - Ans: To analyze the Python project in SonarQube, a new project was created with a unique key, and Python was set as the primary language. An authentication token was generated for secure integration with Jenkins. In Jenkins, the SonarQube Scanner plugin was configured, and analysis parameters like the project key and source directory were added to the pipeline. After triggering the build, the code was scanned, and the results, including code smells and vulnerabilities, were displayed on the SonarQube dashboard.

Conclusion

The deployment of Jenkins and SonarQube on AWS EC2 instances using Terraform exemplifies the power of automation in modern software development and DevOps

practices. Automating infrastructure setup with Terraform ensures consistency, reduces the chances of manual errors, and accelerates deployment processes. By combining Jenkins for Continuous Integration/Continuous Delivery (CI/CD) with SonarQube for continuous code quality analysis, teams can streamline their development pipelines, ensuring that every code change is thoroughly tested and evaluated before release. This seamless integration enables early detection of code smells, bugs, and vulnerabilities, contributing to higher software quality and more maintainable projects over time.

This case study demonstrates how leveraging Infrastructure as Code (IaC) tools like Terraform, in conjunction with modern development tools, offers a robust solution for managing complex software lifecycles. It highlights the importance of continuous feedback loops in improving software quality while accelerating the release cycle, ultimately helping organizations achieve faster time to market, greater agility, and sustainable growth. Adopting such practices is essential for organizations aiming to remain competitive in today's dynamic software development landscape.